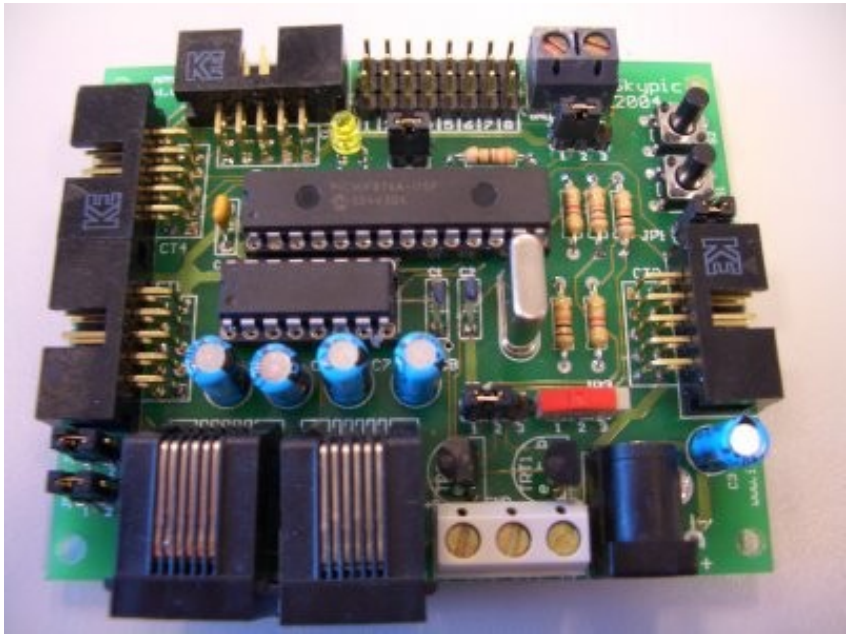
PIC 16F87X



Juan González

Escuela Politécnica Superior
Universidad Autónoma de Madrid

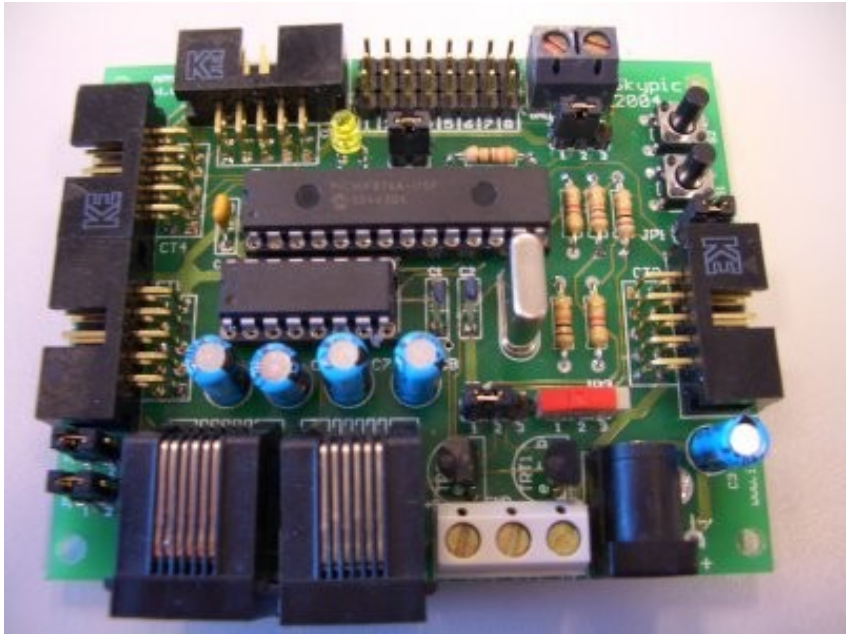
Andrés Prieto-Moreno

Flir Networked Systems

Ricardo Gómez

Flir Networked Systems

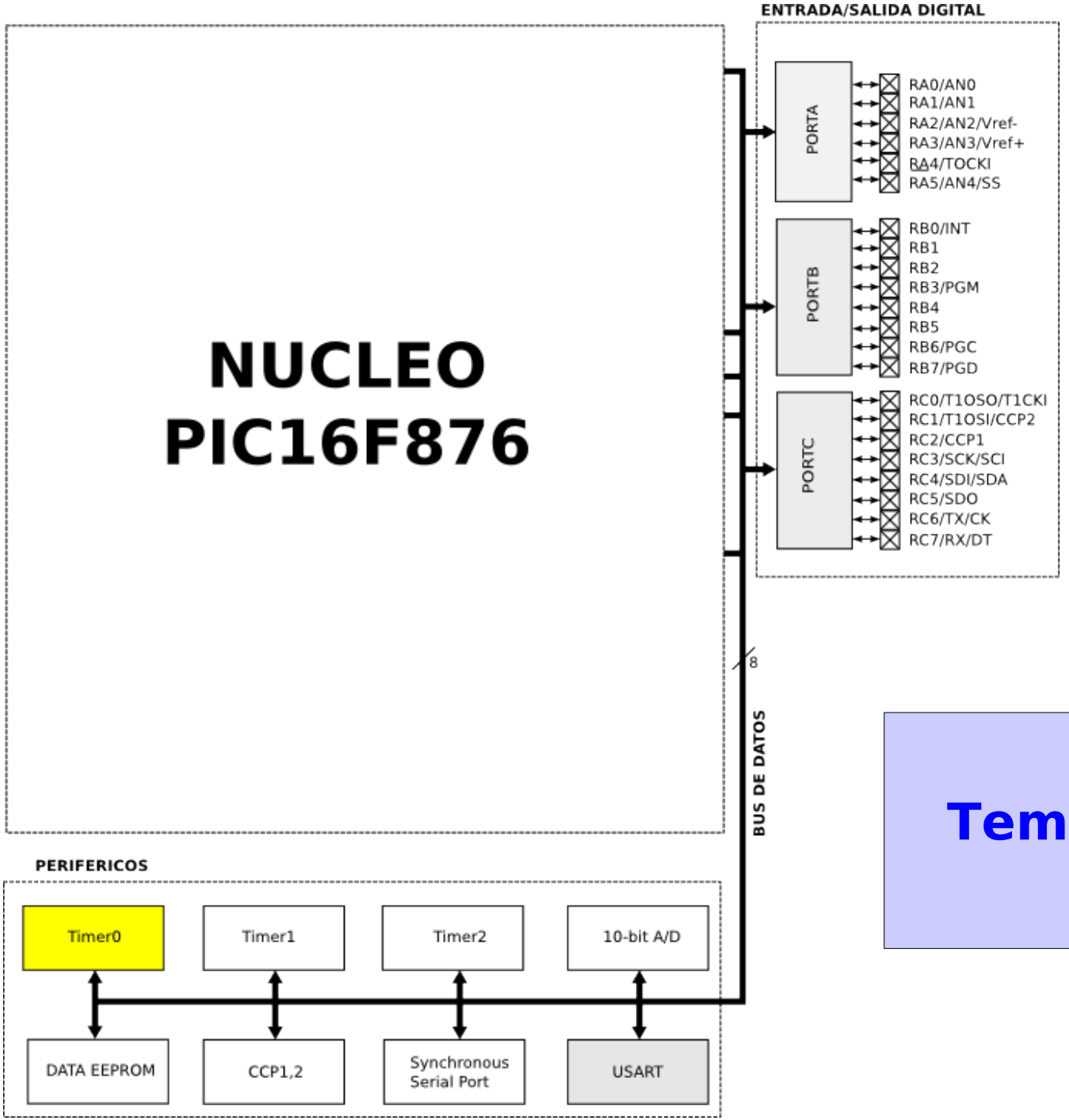
PIC 16F87X



MÓDULO 4:

Temporizadores

Programación de periféricos



Temporizador 0

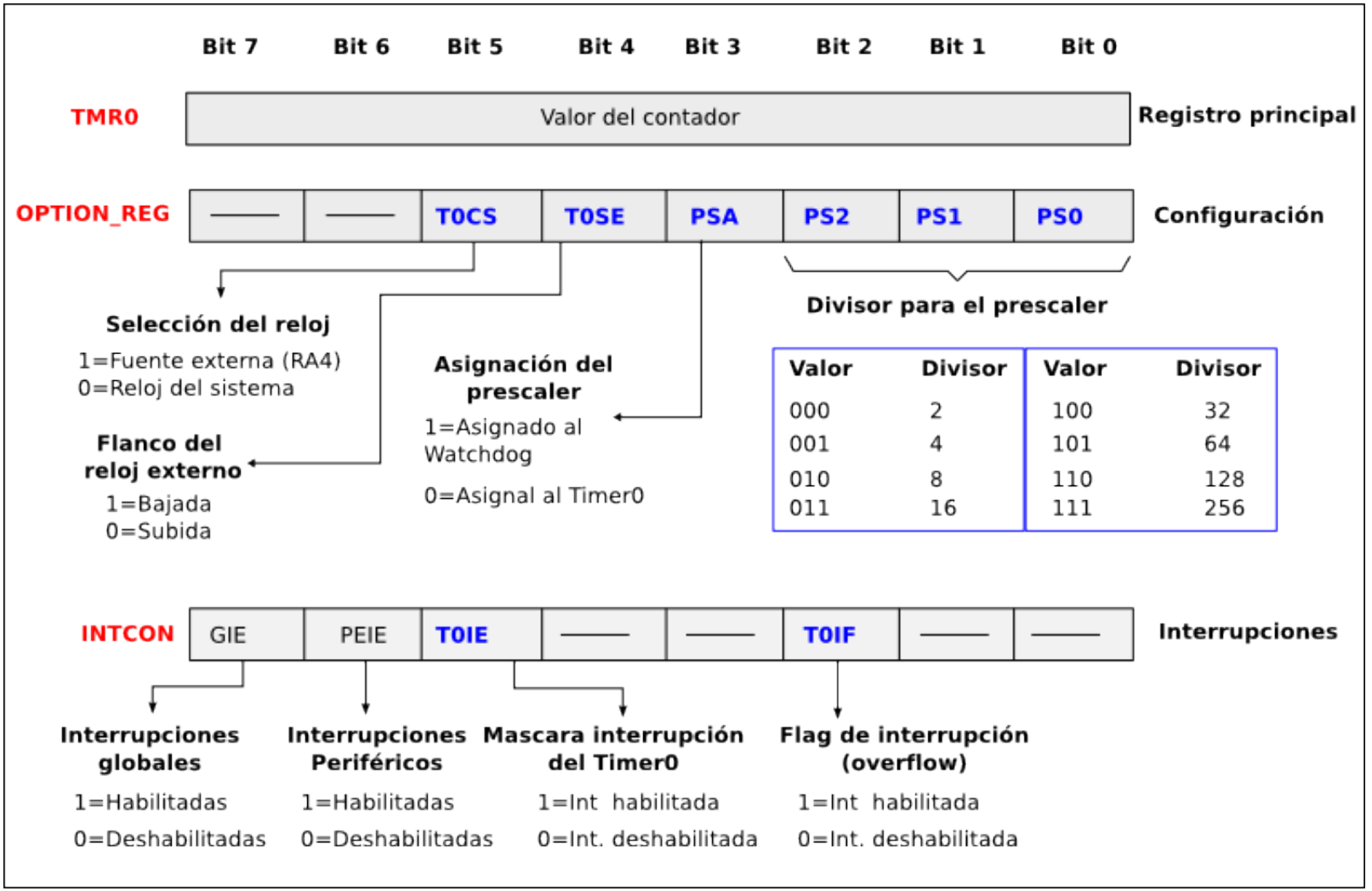
Temporizador 0

- Temporizador de **8 bits**
- **Modos de funcionamiento**:
  - Temporizador (reloj interno)
  - Contador de pulsos externos (configurable para flancos de subida/bajada)
- **Interrupción de overflow** cuando pasa de 0xFF a 0x00

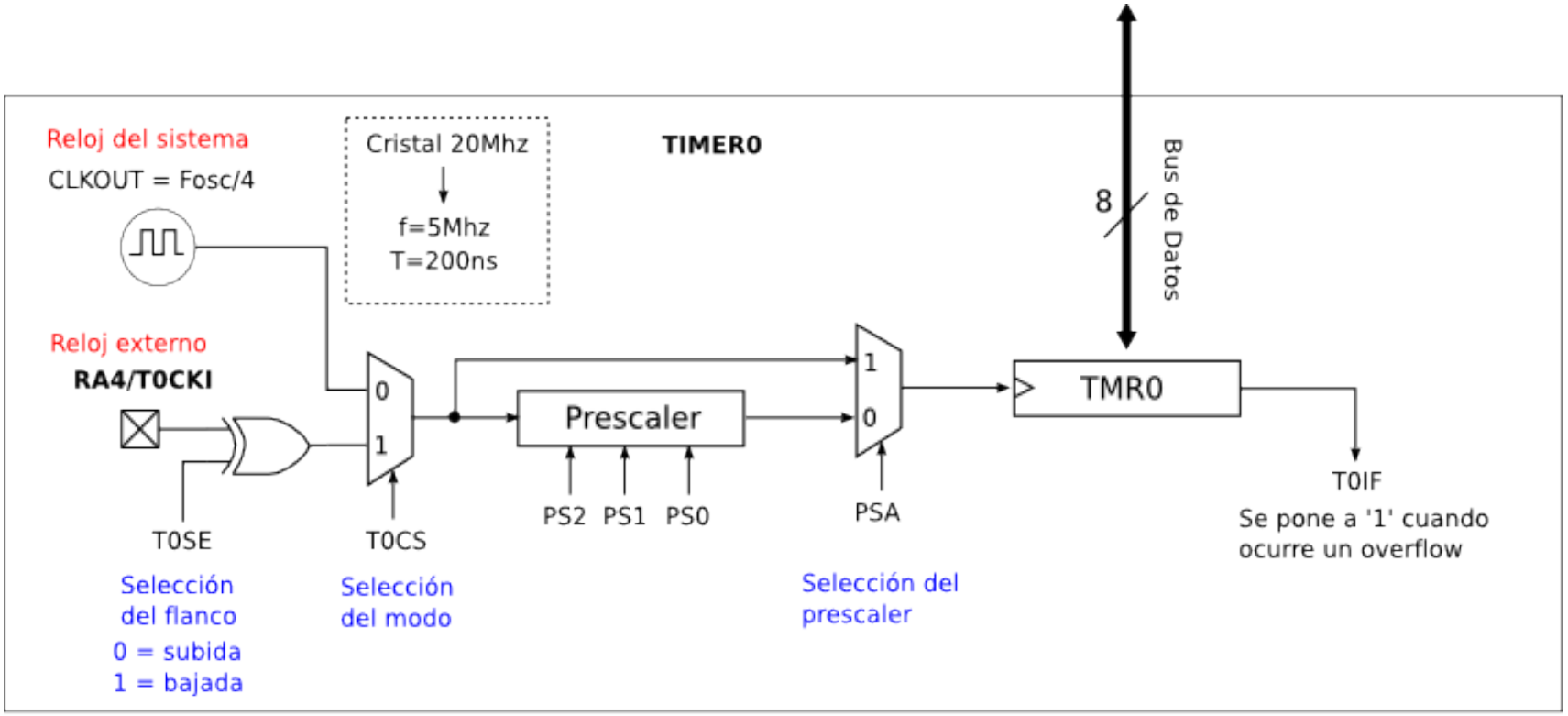
Aplicaciones

- **Modo temporizador**:
  - Rutinas para generar pausas de tiempo conocido
  - Generar ondas cuadradas de diferentes frecuencias (Ej. pitidos, CLKs)
  - Generar señales PWM (Ej. mover motores o servos)
  - Medir intervalos de tiempo (Ej. cronómetros)
- **Modo contador**
  - Contar pulsos (Ej. lectura encoders, contador de sucesos)
  - Cuenta por hardware: el software sólo tiene que leer un registro

Registros del temporizador 0



Funcionamiento del Timer 0



Configuración		
	Modo temporizador:	T0CS=0 PSA=0
	Modo contador:	T0CS=1 PSA=1

- Reloj del sistema = Frecuencia del Cristal/4
- En la Skypic, **Cristal 20Mhz** --> f=5Mhz, T=200ns

Dominios de tiempo

- Tiempo en “ticks”
  - En el PIC, nuestras unidades de tiempo son los **ticks**
  - El temporizador cuenta ticks
  - Ej. TMR0=20 significa que han transcurrido 20 ticks desde que estaba a 0

- Tiempo natural
  - Es el tiempo que usamos en la realidad (ms, us, ns, etc..)
  - La **relación entre tiempo natural y ticks** es:

$$t = N \cdot T_{reloj} \cdot Divisor$$

Valor del prescaler

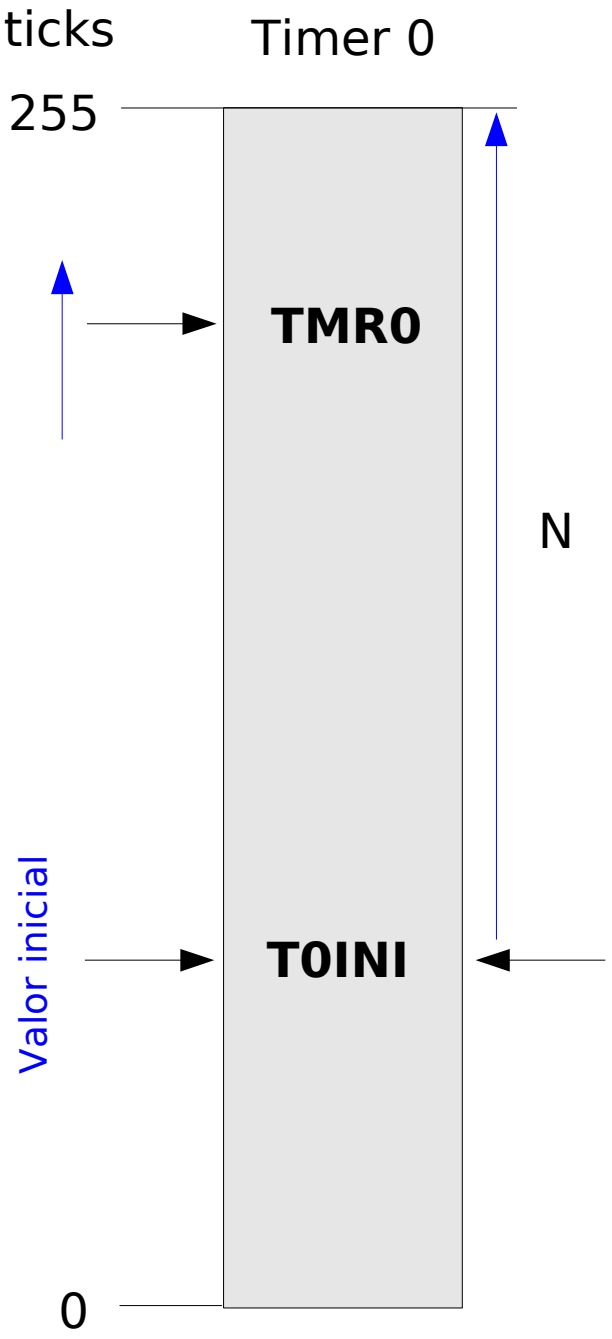
Tiempo natural

Tiempo en ticks

Periodo del reloj del sistema

=200ns en la Skypic

Midiendo el tiempo (I)



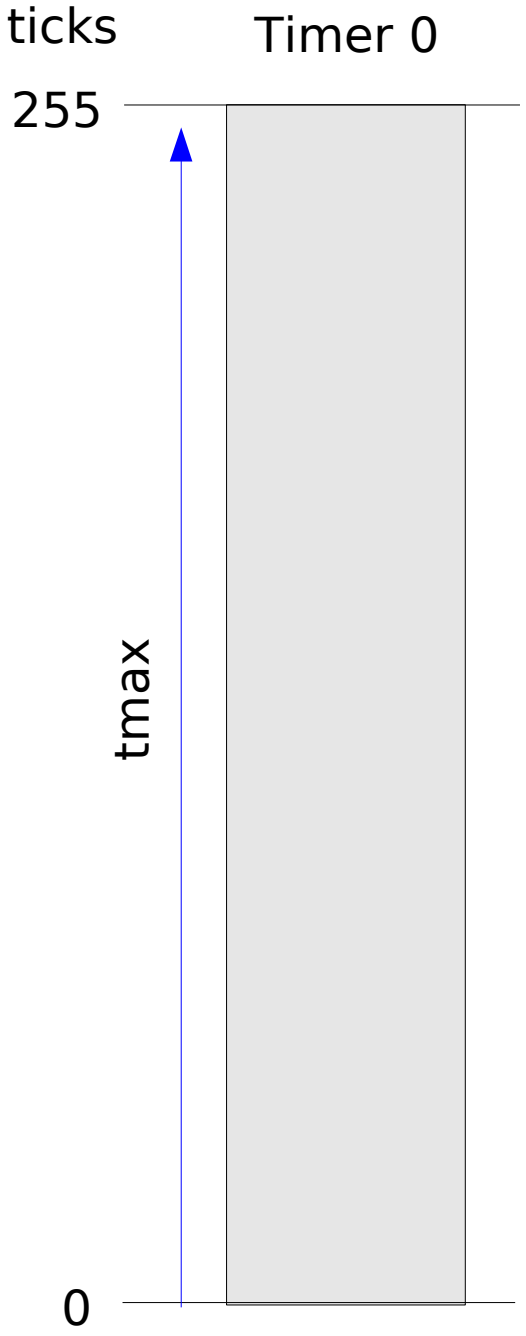
- Para medir tiempo usamos el flag de overflow
- Queremos que el temporizador “nos avise” cuando hayan transcurridos N ticks.
- Para ello inicializamos el temporizador con el valor **T0INI**, que se calcula como:

$$T0INI = 256 - N$$

- Con este valor inicial, al cabo de N ticks se producirá el overflow
- En unidades naturales, el overflow se producirá al cabo de un **tiempo t**, donde t se calcula con la fórmula de la transparencia anterior. Expresada en función de la frecuencia del Cristal es:

$$t = \frac{4}{Fosc} . N . Divisor$$

¿Cuanto tiempo podemos medir?



• El tiempo máximo medible será cuando hayan transcurrido **N=256 ticks** (porque el temporizador es de 8 bits):

$$t_{max}(\mu s) = \frac{1024}{Fosc(Mhz)} . Divisor$$

• El tiempo mínimo medible es el correspondiente a **N=1 tick**:

$$t_{min}(\mu s) = \frac{4}{Fosc(Mhz)} . Divisor$$

• El rango de tiempo “medible” será: (t_{min}, t_{max})

• Como el valor del periodo del reloj del sistema será fijo (está fijado por el cristal usado), este **rango depende del valor del Divisor del prescaler**

Rangos de tiempo para cristal de 20Mhz

• Aplicando las fórmulas anteriores, obtenemos una tabla con el rango de tiempos medibles

• Tiempos fuera de estos intervalos no se pueden medir directamente con el Timer0

Fosc=20Mhz

Divisor	t_{min}	t_{max}
2	0.4 µs	102.4 µs
4	0.8 µs	204.8 µs
8	1.6 µs	409.6 µs
16	3.2 µs	819.2 µs
32	6.4 µs	≃ 1.64 ms
64	12.8 µs	≃ 3.28 ms
128	25.6 µs	≃ 6.6 ms
256	51.2 µs	≃ 13.1 ms

Cálculos para medir tiempo

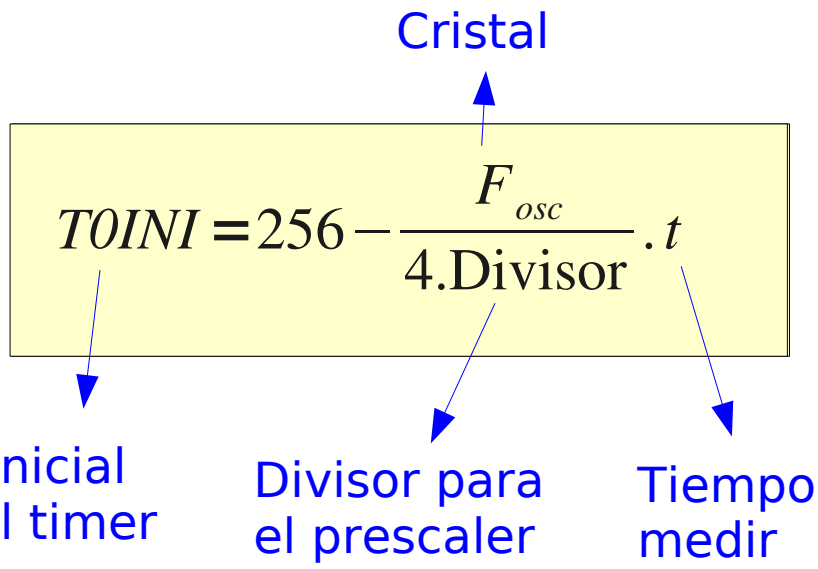
- Queremos medir un tiempo t
- Necesitamos conocer el **valor inicial del Timer 0** (T0INI) para que se active el **flag de overflow** transcurrido ese tiempo t

A partir de las ecuaciones anteriores:

$$t = N \,.\, T_{reloj} \,.\, Divisor = \frac{4}{F_{osc}} \,.\, N \,.\, Divisor$$

T0INI = 256 - N

Se obtiene:



- T0INI es un **número entero**
- Después de aplicar esta fórmula, hay que **redondear**
- **Aparecerá un error**

Error de redondeo

• A partir del tiempo a medir t, obtenemos el valor inicial para el temporizador (T0INI). Este valor estará redondeado.

• A partir de T0INI (que está en ticks) calculamos el tiempo natural hasta que se produzca el overflow. Este valor debería ser t, pero debido al redondeo será diferente. Lo llamaremos tc:

$$t_c = (256 - T0INI) \cdot \frac{4}{F_{osc}} \cdot Divisor$$

• El error debido al redondeo será:

$$\epsilon = t - t_c$$

t → Tiempo que se mide realmente

t_c → Tiempo que se quiere medir

Metodología

- Dato de partida: el tiempo t a medir

1) **Obtener los valores** posibles de los **divisores**, a partir de la **tabla** de los rangos de tiempo

2) Mediante la ecuación, **obtener el valor T0INI** para cada uno de los **divisores** encontrados

$$T0INI = 256 - \frac{F_{osc}}{4.\text{Divisor}}.t$$ (Redondeado)

3) Todos los pares (T0INI,Divisor) obtenidos serán **soluciones válidas**. Sin embargo, debido al **error de redondeo** unas serán mejores que otras.

4) (opcional) **Calcular el error de redondeo** y seleccionar como solución el que lo tenga más bajo (opcional)

$$t_c = (256 - T0INI).\frac{4}{F_{osc}}.Divisor$$

$$\epsilon = t - t_c$$

Ejemplo. Pausa de 1 ms (I)

¿Cómo hacer una pausa de 1ms?

Dato de partida, t=1ms

1) **Obtener los valores** posibles de los **divisores**, a partir de la **tabla** de los rangos de tiempo

Divisor	t_{min}	t_{max}
2	0.4 µs	102.4 µs
4	0.8 µs	204.8 µs
8	1.6 µs	409.6 µs
16	3.2 µs	819.2 µs
32	6.4 µs	≃ 1.64 ms
64	12.8 µs	≃ 3.28 ms
128	25.6 µs	≃ 6.6 ms
256	51.2 µs	≃ 13.1 ms

Divisores válidos:
32,64,128,256

Ejemplo. Pausa de 1 ms (II)

2) Mediante la ecuación, **obtener el valor T0INI** para cada uno de los **divisores** encontrados

Divisor	T0INI
32	99.75 ≃ 100
64	177.88 ≃ 178
128	216.94 ≃ 217
256	236.47 ≃ 236

3) Todos los pares (T0INI,Divisor) obtenidos serán **soluciones válidas**. Sin embargo, debido al **error de redondeo** unas serán mejores que otras.

Ejemplo. Pausa de 1 ms (III)

4) (opcional) **Calcular el error de redondeo** y seleccionar como solución el que lo tenga más bajo (opcional)

Divisor	T0INI	t	t_c	$\|\varepsilon\|$
32	100	1 ms	998.4 µs	1.6 µs
64	178	1 ms	998.4 µs	1.6 µs
128	217	1 ms	998.4 µs	1.6 µs
256	236	1 ms	1024 µs	24 µs

Cualquiera de esas soluciones es igual de buena

- **Elegimos una**. Por ejemplo esta: Divisor = 32, T0INI = 100

Ejemplo. Pausa de 1 ms (IV)

timer0-pausa1ms.c

```
#include <pic16f876a.h>
#define LED RB1
#define TOINI 100

void pausa_1ms()
{
  TMR0=TOINI;
  T0IF=0;
  while(T0IF==0);
}
void main(void)
{
  TRISB1=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=0; PS0=0;
  while(1) {
    pausa_1ms();
    LED=1;
    pausa_1ms();
    LED=0;
  }
}
```

Valor inicial del timer para conseguir pausas de 1ms

Inicializar timer

Poner flag de overflow a 0

Esperar a que transcurra 1ms. Al terminar se activa el flag de overflow

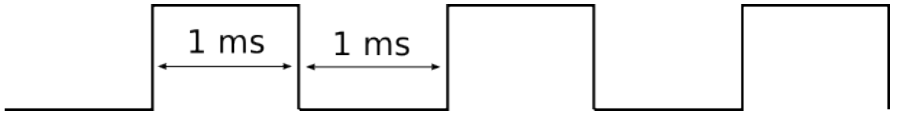
Configurar timer 0 en modo temporizador

Configurar el divisor del prescaler a 32

Enviar una señal al led

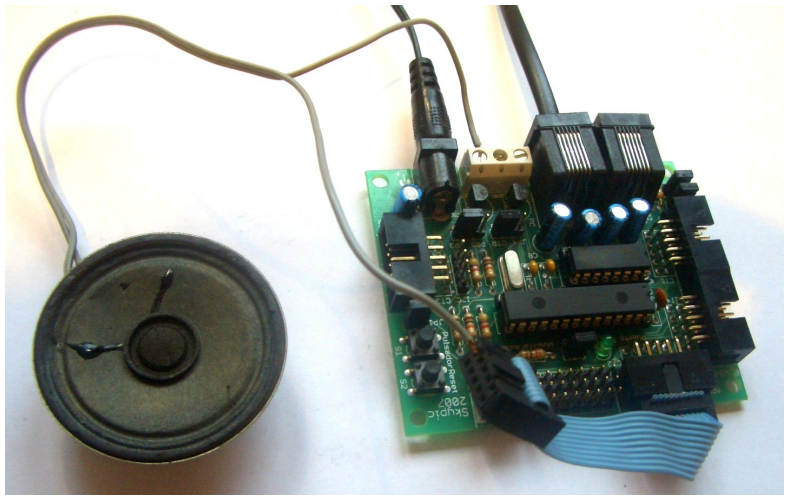
Ejemplo. Pausa de 1 ms (V)

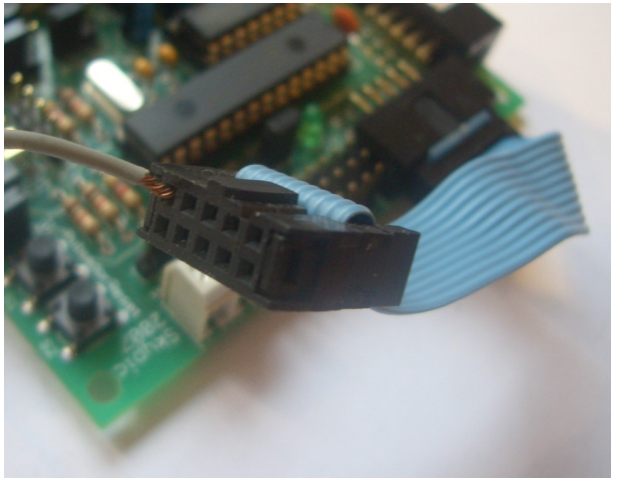
- Este programa genera una **señal cuadrada** que sale por RB1



- La frecuencia de la señal es de 500Hz por lo que no es apreciable por el ojo humano y no se verá cómo parpadea el led

- Sin embargo si conectamos un altavoz oiremos un tono:

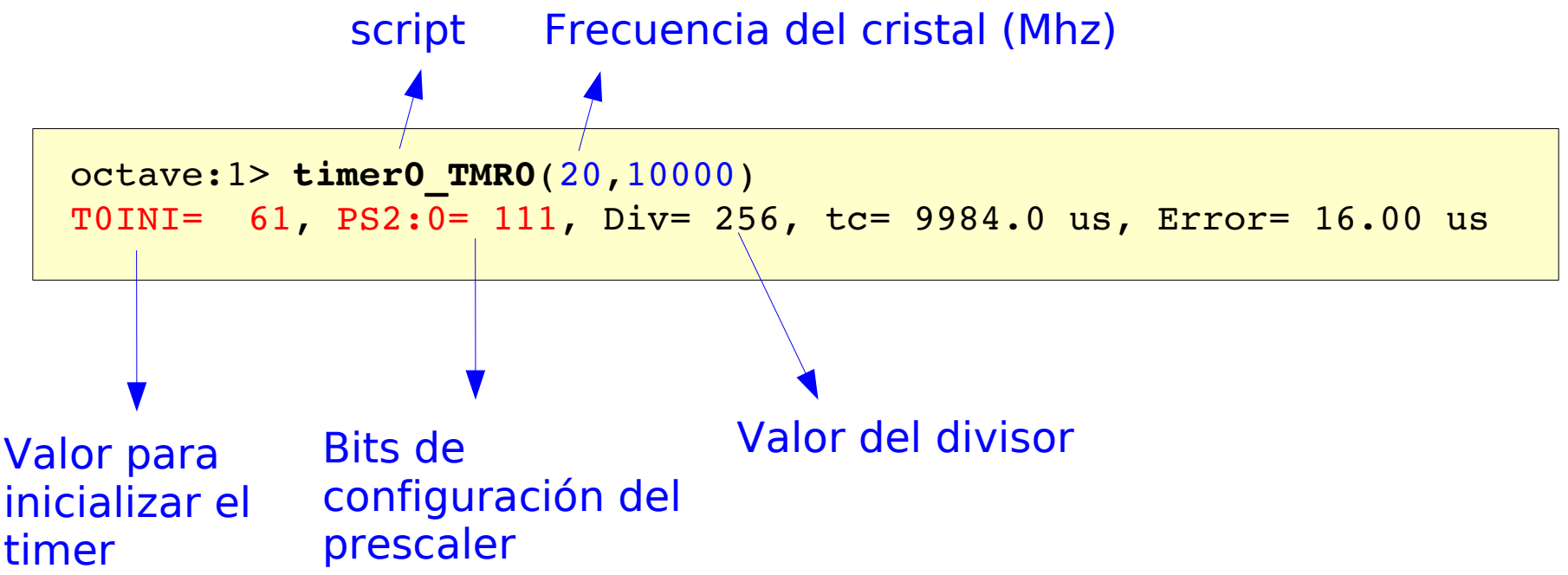




Ejemplo. Pausa de 10 ms (I)

- Dato de partida, t=10ms
- Hay que **repetir los cálculos** para el nuevo valor de **t**
- Estas operaciones repetitivas es mejor **automatizarlas**

- **Script en Octave/Matlab** para calcular los valores de T0INI y el divisor



- En este ejemplo, la solución es única: T0INI=61, Divisor=256

Ejemplo. Pausa de 10 ms (II)

timer0-pausa10ms.c

```
#include <pic16f876a.h>
#define LED RB1
#define T0INI 61

void pausa_10ms()
{
  TMR0=T0INI;
  T0IF=0;
  while(T0IF==0);
}
void main(void)
{
  TRISB1=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=1; PS0=1;
  while(1) {
    pausa_10ms();
    LED=1;
    pausa_10ms();
    LED=0;
  }
}
```

- El ejemplo es igual que el anterior, pero cambiando los valores de configuración del timer

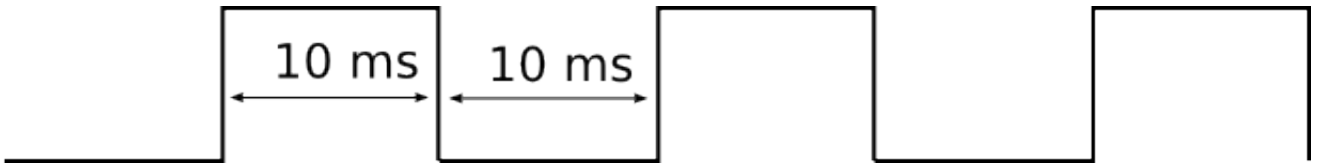
Valor inicial del timer para conseguir pausas de 10ms

Otro nombre a la función

Nuevo valor del divisor: 256

Ejemplo. Pausa de 10 ms (III)

• Ahora se genera una **señal cuadrada** de 50Hz



• Sigue sin ser visible el parpadeo del led, pero el tono es audible, aunque más grave que en el caso el ejemplo anterior

Ejercicio:

- Hacer un Programa que genere una pausa de 500 micro-segundos.

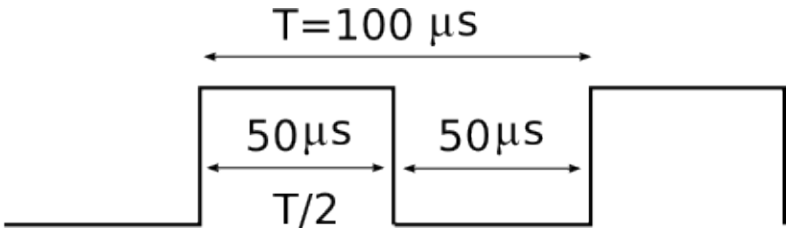
- Calcular todos los pares de valores: Divisor, T0ini para generar esa pausa

- Comprobar que con todos ellos se genera la misma pausa. Para ello generar una señal cuadrada de 1ms y comprobar que suena igual al usar los diferentes valores.

Ejemplo. Señal de 10Khz (I)

• Ejemplo similar a los anteriores. Se aplica el mismo método, pero tomando ***t=T/2***

$$t = \frac{T}{2} = \frac{1}{2.f}$$



• **Script en Octave/Matlab** para facilitarnos la vida:

Frecuencia del cristal (Mhz)

script

Frecuencia señal cuadrada (Hz)

```
octave:3> timer0 TMR0 frec(20,10000)
TMR0= 131, PS2:0= 000, Div=   2, Frec= 10000.0 Hz, Error= 0.00 Hz
TMR0= 193, PS2:0= 001, Div=   4, Frec= 9920.6 Hz, Error= 79.37 Hz
TMR0= 225, PS2:0= 010, Div=   8, Frec= 10080.6 Hz, Error= -80.65 Hz
TMR0= 240, PS2:0= 011, Div=  16, Frec= 9765.6 Hz, Error= 234.38 Hz
TMR0= 248, PS2:0= 100, Div=  32, Frec= 9765.6 Hz, Error= 234.38 Hz
TMR0= 252, PS2:0= 101, Div=  64, Frec= 9765.6 Hz, Error= 234.38 Hz
TMR0= 254, PS2:0= 110, Div= 128, Frec= 9765.6 Hz, Error= 234.38 Hz
TMR0= 255, PS2:0= 111, Div= 256, Frec= 9765.6 Hz, Error= 234.38 Hz
```

• Varias soluciones. Tomamos: T0INI=131, Divisor=2

Ejemplo. Señal de 10Khz (I)

timer0-onda10Khz.c

```
#include <pic16f876a.h>
#define T0INI 131
#define LED RB1

void timer0_delay(unsigned char t0ini)
{
  TMR0=t0ini;
  T0IF=0;
  while(T0IF==0);
}
void main(void)
{
  TRISB1=0;
  T0CS=0; PSA=0;
  PS2=0; PS1=0; PS0=0;
  while(1) {
    timer0_delay(T0INI);
    LED=1;
    timer0_delay(T0INI);
    LED=0;
  }
}
```

Valor inicial para conseguir los semiperiodos de las señales de 10Khz

Función de pausa “genérica”, a la que se le pasa como parámetro el valor inicial del timer. Esto permitirá **reutilizarla para generar señales de diferentes frecuencias**

Divisor a 2

Primer semiperiodo: Señal a nivel alto

Segundo semiperiodo: Señal a nivel bajo

Ondas de distintas frecuencias: Notas musicales (I)

- Vamos a generar **ondas cuadradas de diferentes frecuencias**.
- Como no tenemos osciloscopio para verificar su funcionamiento, usaremos nuestro oído ;-)
- Rango de frecuencias audibles: **20Hz - 20Khz**

- **Notas musicales**:

Nota	Frecuencia (Hz)
DO	261
RE	294
MI	330
FA	349
SOL	392
LA	440
SI	494
DO	261*2
...	...

Octava 1

Octava 2

La frecuencia de la siguiente octava es el doble que la anterior

Ondas de distintas frecuencias: Notas musicales (II)

- Utilizando los métodos anteriores (fórmulas o scripts de octave) se calculan los valores del temporizador y del divisor.

- Se han seleccionado el **divisor que es común** a todas las notas de la primera escala: 64

Nota	Frecuencia (Hz)	Divisor	T0INI
DO	261	64	106
RE	294	64	123
MI	330	64	138
FA	349	64	144
SOL	392	64	156
LA	440	64	167
SI	494	64	177
DO	261*2	64	181

Ondas de distintas frecuencias: Notas musicales (III)

timer0-escala.c (parte I)

```
#include <pic16f876a.h>
#define LED RB1
#define DO  106
#define RE  123
#define MI  138
#define FA  144
#define SOL 156
#define LA  167
#define SI  177
#define DO_A 181
void timer0_delay() {...}
void nota(unsigned char n)
{
  unsigned char i;

  for (i=0; i<100; i++) {
    timer0_delay(n);
    LED=1;
    timer0_delay(n);
    LED=0;
  }
}
```

Valores iniciales del temporizador para generar las frecuencias de las diferentes notas musicales

Función definida en los ejemplos anteriores

Función para tocar una nota

Bucle para generar un tren de 100 ondas. Esto define la duración de la nota

Transcurrido el semiperiodo poner señal a 1

Transcurrido el semiperiodo poner señal a 0

Ondas de distintas frecuencias: Notas musicales (IV)

timer0-escala.c (parte II)

```
void main(void)
{
  TRISB1=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=0; PS0=1;

  nota(DO);
  nota(RE);
  nota(MI);
  nota(FA);
  nota(SOL);
  nota(LA);
  nota(SI);
  nota(DO_A);

  while(1);
}
```

- Ejemplo que toca la escala musical

Configurar el divisor a 64

Tocas las diferentes notas

Pausas de “larga duración” (I)

- **Tiempo máximo** que el Timer 0 puede contar: 13.1 ms

¿Cómo medir tiempos más largos?

- Hay que definir una **nueva unidad de ticks** y **programar un contador software** que se incremente con estos nuevos ticks hasta alcanzar el tiempo deseado.

- Por ejemplo, se puden usar ticks de 10 ms. Cada vez que transcurran esos 10 ms, se incrementará nuestro contador software.

- El tiempo máximo dependerá del número de bits empleados para este contador

timer0-delay.c

Pausas de “larga duración” (II)

```
#include <pic16f876a.h>
#define T_10ms 61
#define LED RB1
void timer0_delay() {...}
void timer0_delay_long(unsigned int duracion)
{
  unsigned int i;

  for (i=0; i<duracion; i++)
    timer0_delay(T_10ms);
}
void main(void)
{
  TRISB1=0;
  LED=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=1; PS0=1;
  while(1) {
    LED^=1;
    timer0_delay_long(50);
  }
}
```

Valor para generar ticks de 10 ms

Función definida en ejemplos anteriores

Nueva función de pausa “larga”. Se para como parámetro un número de 16 bits, que indica la cantidad de ticks de 10 ms a esperar.

Contador de ticks de 10 ms

Pausa de 10 ms (un tick)

Divisor: 256

Cambiar led de estado

Pausa de medio segundo

Ejemplo que enciende el led cada segundo

Pausas de “larga duración” (III)

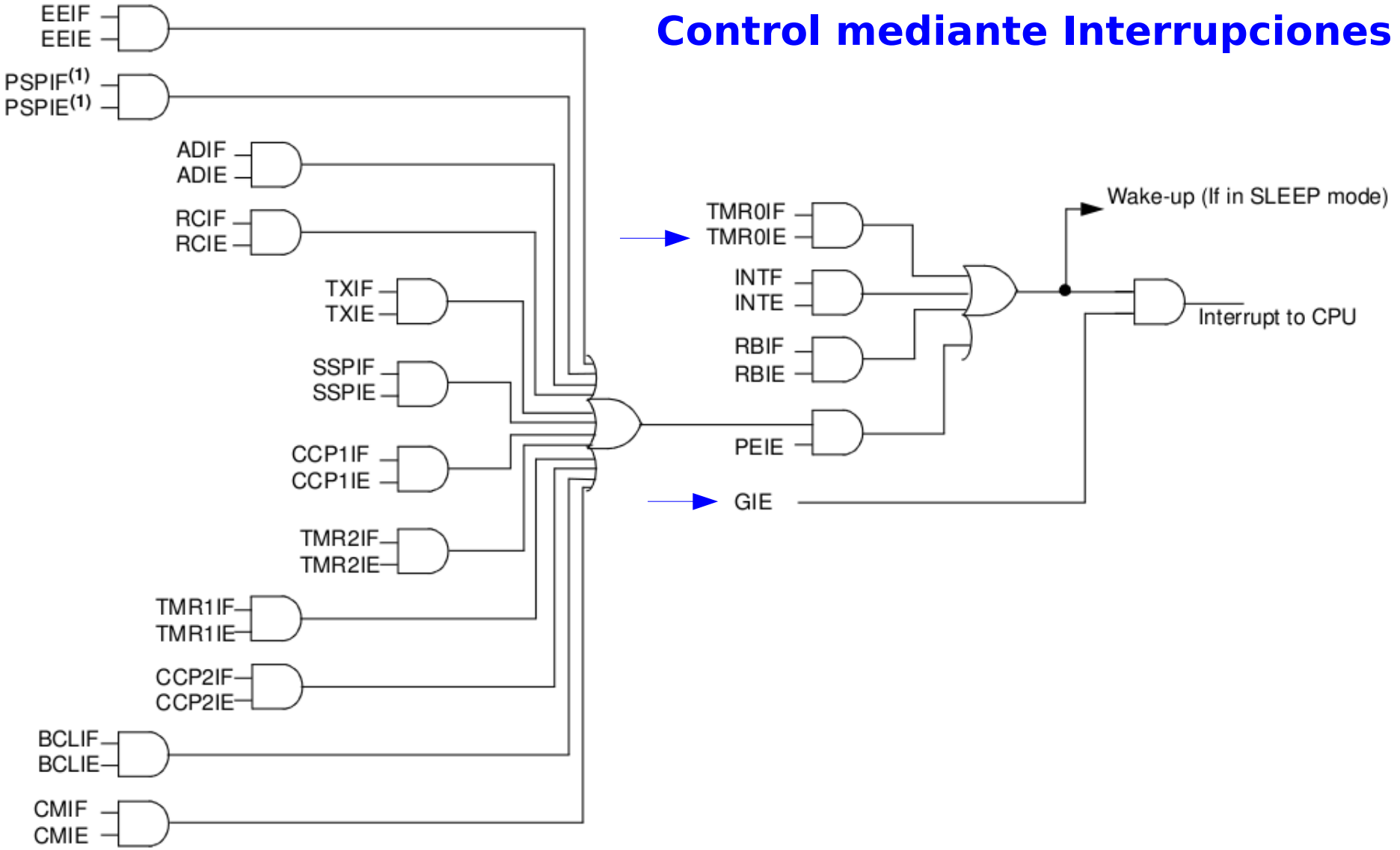
- El **Tiempo máximo** que se puede esperar será:

  Tiempo máximo = Número ticks x Duracion ticks

- Para el ejemplo, se usa un contador de ticks de 16 bits (unsigned int).
- El tiempo máximo será:

  Tmax = 65535 x 10ms = 655350ms --> 655 seg (10.9 min)

Control mediante Interrupciones



Note 1: PSP interrupt is implemented only on PIC16F874A/877A devices.

Ondas cuadradas e interrupciones (I)

- Las **ondas cuadradas** se pueden generar **mediante interrupciones**
- De esta manera, el PIC puede estar haciendo otras tareas a la vez que se emiten los tonos
- Para **activar las interrupciones** del timer 0 se activan los siguientes bits: GIE=1, TMR0IE=1

- El siguiente ejemplo genera **8 tonos simultáneamente** por los bits del **puerto B**, mediante interrupciones
- Se usa un contador como prescaler de manera que la frecuencia de RB0 sea de 10Khz, la de RB1 de 5Khz, la de RB2 de 2.5Khz, etc..

Ondas cuadradas e interrupciones (II)

timer0-onda10Khz-int.c

```
#include <pic16f876a.h>
#define T_10Khz 131

unsigned char cont=0;
void isr() interrupt 0
{
  cont++;
  PORTB=cont;
  TMR0=T_10Khz;
  TMR0IF=0;
}
void main(void)
{
  TRISB=0;
  T0CS=0; PSA=0;
  PS2=0; PS1=0; PS0=0;
  TMR0IE=1;
  GIE=1;
  while(1);
}
```

Valor para generar semiperiodos correspondientes a una señal de 10Khz

Contador que se incrementa con una frecuencia de 10Khz. El bit menos significativo tendrá la máxima frecuencia (10Khz) y el resto dividid por su peso

Rutina de atención a la interrupción. Se ejecuta en cada semiperiodo de la señal de 10Khz

Incrementar contador y enviarlo por el puerto B

Configurar timer para el siguiente semiperiodo. Quitar el flag de interrupción

Divisor del prescaler a 2

Activar interrupción de overflow e interrupciones globales

El bucle principal no hace nada

Según el bit del puerto B donde se conecte el altavoz se oirá una frecuencia u otra. En total 8 diferentes.

Órgano digital (I)

• Al generarse los tonos mediante interrupciones, **el PIC puede realizar otras tareas en paralelo**.

• El siguiente programa reproduce las notas musicales según las teclas que recibe desde el Pc, mediante un terminal de comunicaciones

• La rutina de interrupciones se encarga de generar los tonos

• El bucle principal procesa los caracteres que llegan por el SCI

• La **comunicación** entre las dos tareas se hace mediante **variables globales**.



¿Quién dijo que programar PICs era aburrido? :-)

Órgano digital (II)

timer0-organo.c (parte I)

```
#include <pic16f876a.h>
#include "sci.h"
#define DO  106
#define RE  123
#define MI  138
#define FA  144
#define SOL 156
#define LA  167
#define SI  177
#define DO_A 181

unsigned char nota[]= {DO,RE,
  MI,FA,SOL,LA,SI,DO_A};
unsigned char cont=0;
unsigned char tono;

void isr() interrupt 0
{
  cont++;
  PORTB=cont;
  TMR0=tono;
  TMR0IF=0;
}
```

Incluir el módulo del SCI (por el método “cutre”)

Notas musicales. Las mismas que las del ejemplo de la escala

Tabla con las notas. nota[0] es el DO, nota[1] el RE, etc.

Contador para hacer el órgano “politono”. Igual que en el ejemplo anterior

Variable global que contiene el tono actual a tocar

Rutina de atención a la interrupción. Es la que toca los tonos. Igual que la del ejemplo anterior

El tono que se toca no es siempre el mismo (como en el ejemplo anterior) sino que será el que indique la variable global tono.

Órgano digital (III)

```
void main(void)
{
  unsigned char i,c;
  TRISB=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=0; PS0=0;
  sci_conf();
  menu();
  GIE=1;

  while(1) {
    c=sci_read();
    if (c>='1' && c<='8') {
      i=c-'1';
      tono=nota[i];
      TMR0IE=1;
    }
    else TMR0IE=0;
  }
}
```

Divisor a 32

Configurar puerto serie

Sacar menu de opciones (no mostrado en las transparencias)

Activar las interrupciones globales, pero todavía no la del timer. Sólo queremos que suene cuando el usuario apriete las teclas válidas

Esperar a que el usuario pulse una tecla

Sólo son válidas las teclas del 1 al 8

Obtener el índice a la nota. La tecla '1' tiene el índice 0 y se corresponde con la nota DO

Poner en la variable global tono la nota a tocar

Activar interrupción del Timer 0!! Empieza a sonar!!

Una tecla no valida desactiva el sonido

Ejercicio:

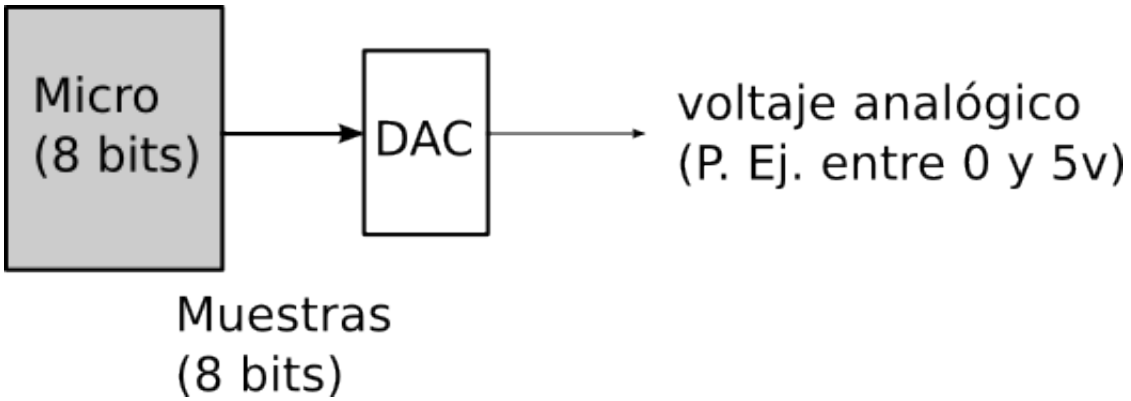
• Hacer un programa que **toque la escala musical**, pero que entre nota y nota haga una **pausa exacta de medio segundo**

Generación de señales PWM (I)

• **Los microcontroladores son digitales**, por lo que funcionan con sólo dos niveles de tensiones

• ¿Cómo se puede con ellos controlar **sistemas analógicos**?

Opción 1: Conversores analógico-digitales (DAC)



• Control de motores

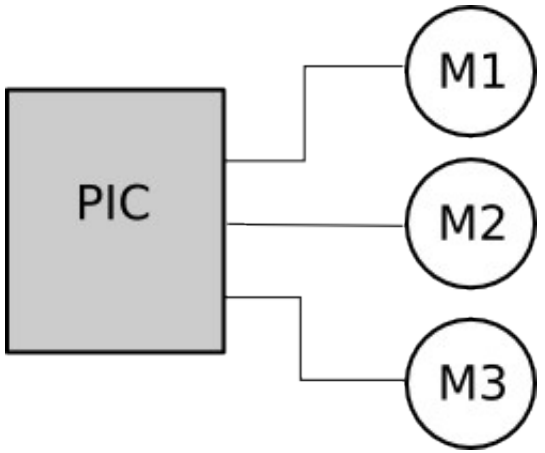
• Aplicaciones de audio

• Etc...

Generación de señales PWM (II)

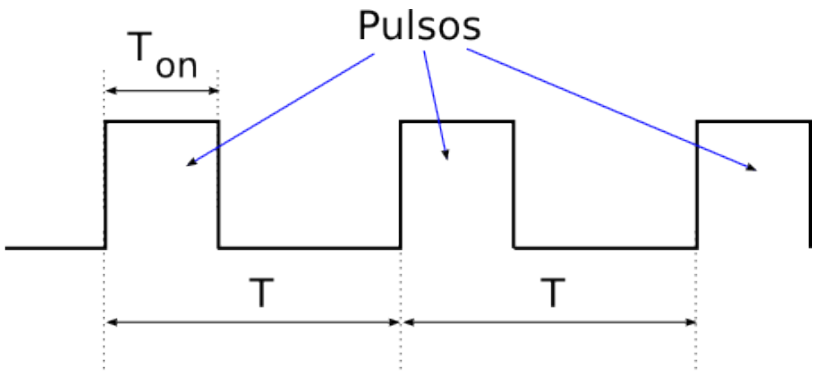
Opción 2: Usar señales **PWM** (*Pulse width modulation*)

- Se pueden controlar sistemas analógicos **usando sólo 1 pin**
- La generación de ondas cuadradas es sencilla en los microcontroladores
- Una de las aplicaciones más usadas es el **control de motores**
- También para el posicionamiento de servos



Principio de funcionamiento (I)

• Generar ondas cuadradas de **frecuencia fija**, pero variando la anchura del pulso



• La señal tiene un periodo T y un ancho del pulso Ton

• Para **frecuencias** mayores de una determinada, los sistemas responden sólo al valor medio de la señal

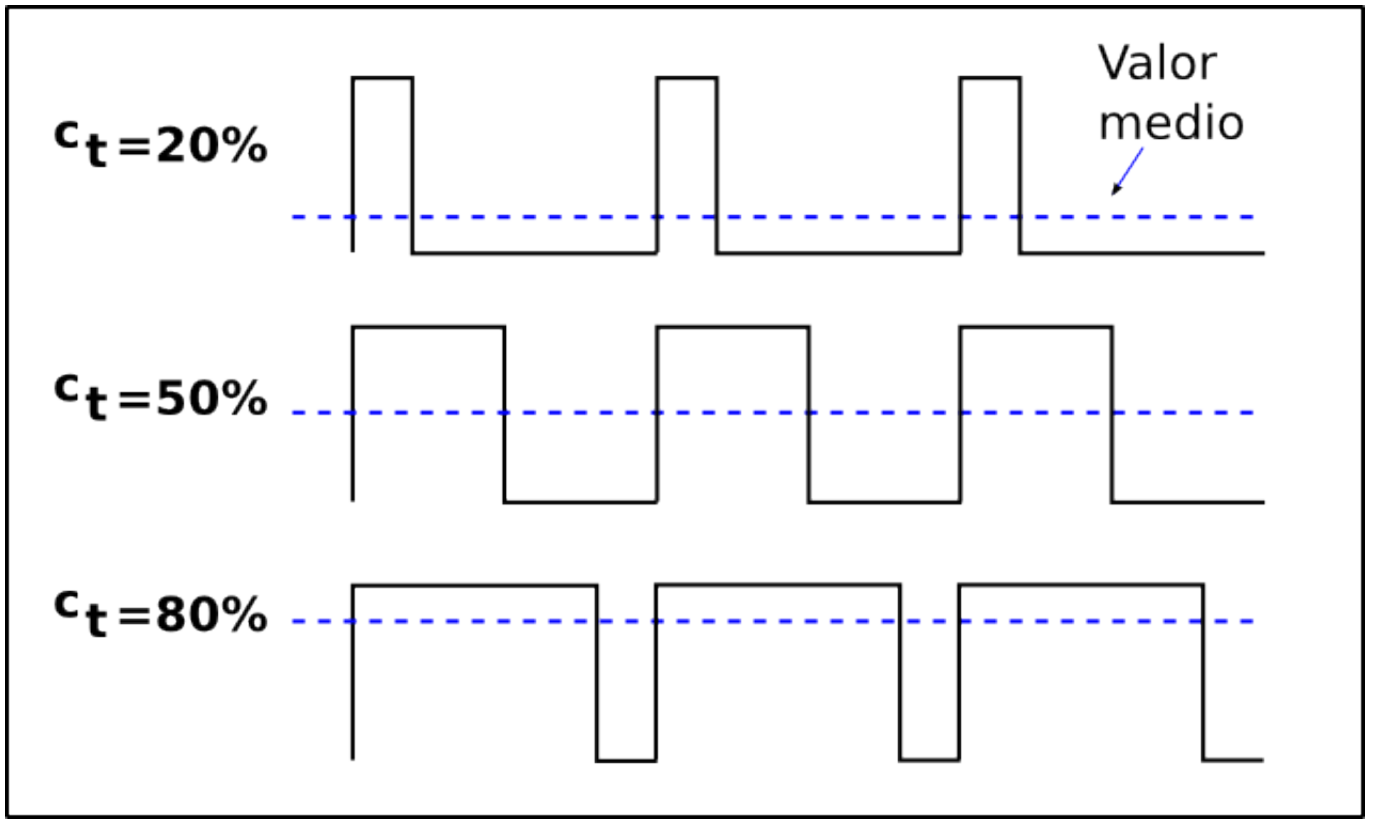
• Ejemplo con leds o luces: Si la frecuencia es de 1Hz, veremos parpadear el led. A partir de 50Hz nuestro ojo ya no lo percibirá: se quedará sólo con el valor medio.

• Ejemplo con motores. A partir de frecuencias de decenas de hercios, la inercia mecánica de los motores les impide responder a instantánemente. Sólo lo harán al valor medio.

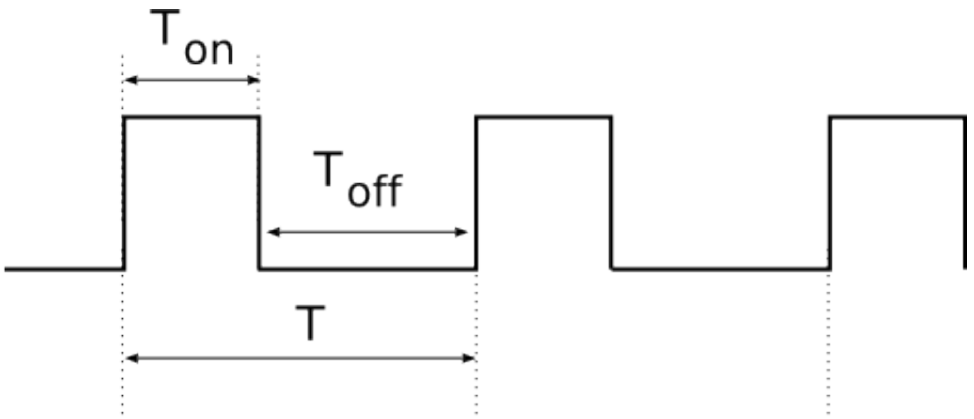
Principio de funcionamiento (II)

• **Ciclo de trabajo**: Porcentaje de tiempo que la señal está activa con respecto al periodo

$$c_t = \frac{T_{on}}{T}$$



Timer0 y PWM



- Generar estas señales es sencillo con el timer
- Primero se calculan los **valores iniciales** del timer y el **divisor** del prescaler para generar pausas de tiempo Ton y Toff (como en los ejemplos anteriores)
- El esquema en pseudocódigo es:

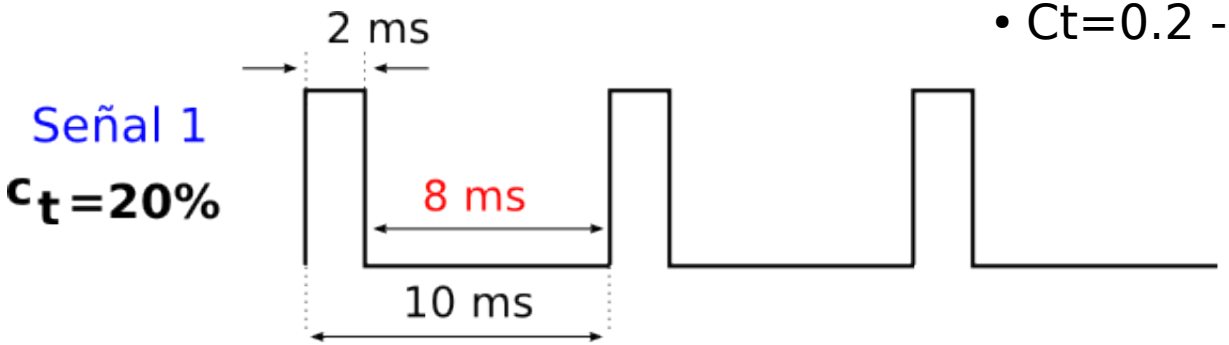
```
Poner señal a 1
Esperar tiempo igual a Ton
Poner señal a 0
Esperar tiempo igual a Toff
```

Toff = T - Ton

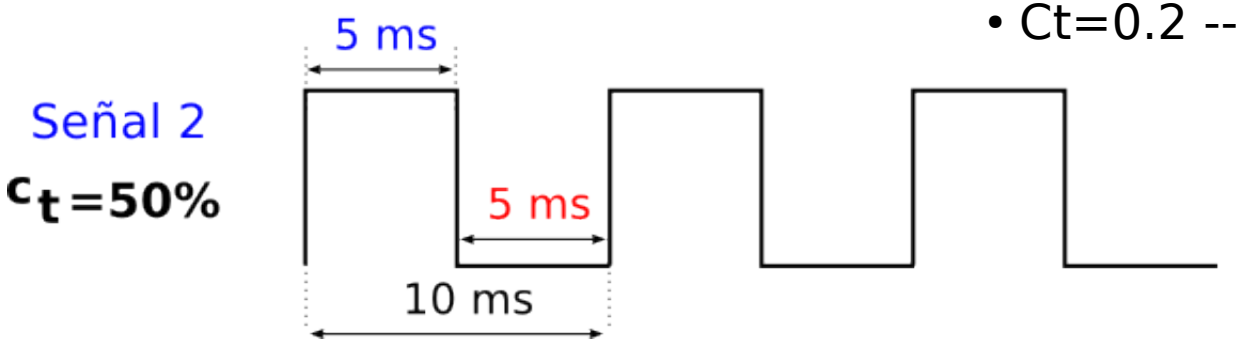
Timer0 y PWM: Ejemplo sencillo (I)

Generar una señal PWM de **100Hz**, con ciclos de trabajo del **20%** y del **50%**

- f=100Hz --> T=10ms



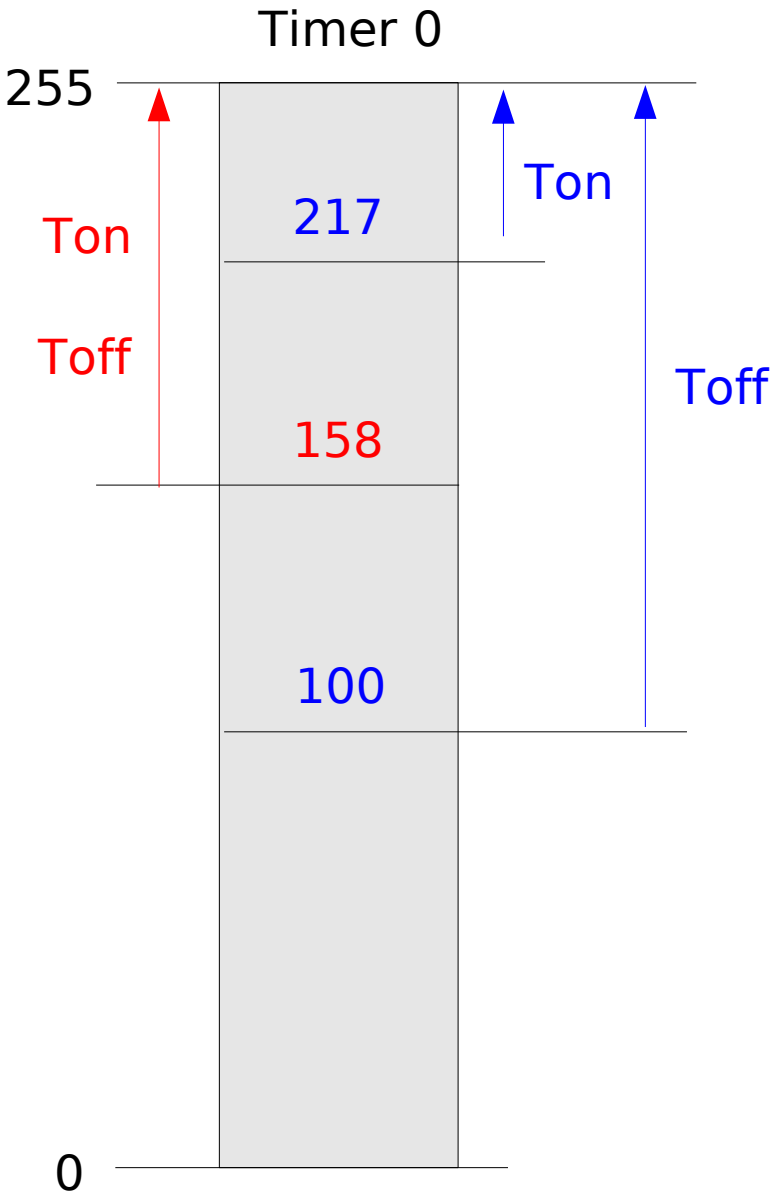
- Ct=0.2 --> Ton=2 ms, Toff=8ms



- Ct=0.2 --> Ton=2 ms, Toff=8ms

Timer0 y PWM: Ejemplo sencillo (II)

- **Realizando cálculos**, se obtienen los siguientes valores:



- **Divisor=256**
- **Señal 1**:
  - Ton: T0INI=217
  - Toff: T0INI=100
- **Señal 2**:
  - Ton: T0INI=158
  - Toff: T0INI=158

Timer0 y PWM: Ejemplo sencillo (III)

timer0-pwm1.c (parte I)

```
#include <pic16f876a.h>

#define TON_C20  217
#define TOFF_C20 100
#define TON_C50  158
#define TOFF_C50 158

unsigned char Ton[]= {TON_C20,  TON_C50};
unsigned char Toff[]={TOFF_C20, TOFF_C50};

void timer0_delay() {...}

void senal(unsigned char i)
{
  unsigned int ciclos;

  for (ciclos=0; ciclos<100; ciclos++) {
    PORTB=0xFF;
    timer0_delay(Ton[i]);
    PORTB=0x00;
    timer0_delay(Toff[i]);
  }
}
```

Constantes con los valores iniciales del timer para las diferentes señales

Tablas con los valores iniciales del timer para los tiempos Ton y Toff de las dos señales

Función utilizada en ejemplos anteriores

Función para generar la señal PWM. Se pasa como parámetro la señal a generar (0 ó 1)

Generar 100 ciclos en la señal para que se pueda ver en los leds

Poner todos los leds a ON

Esperar tiempo Ton de la señal i

Poner leds a off

Esperar tiempo Toff de la señal i

Timer0 y PWM: Ejemplo sencillo (IV)

timer0-pwm1.c (parte II)

```
void main(void)
{
  TRISB=0;
  TOCS=0; PSA=0;
  PS2=1; PS1=1; PS0=1;

  while(1) {
    senal(0);
    senal(1);
  }
}
```

Divisor a 256

Generar la señal de ciclo de trabajo del 20%

Generar la señal de ciclo de trabajo del 50%

• Este ejemplo genera una señal PWM con ciclo del trabajo del 20% sobre todos los leds, luego la cambia por otra del 50% y se repite. El resultado es que se verá que los leds cambian de brillo.

Timer0 y PWM: Ejemplo mejorado (I)

- El ejemplo anterior es sencillo de entender. Como un primer ejemplo nos vale
  - **INCONVENIENTE**:
    - **No es ampliable**. Si queremos generar señales con otros ciclos de trabajo hay que estar recalculando los valores del timer correspondientes a Ton y Toff y añadiéndolos a las tablas.
- Versión mejorada:
  - Es todo más sencillo si se trabaja con T y Ton **expresados en ticks**, en vez de trabajar con los valores de inicialización del timer (T0INI)
  - El valor del timer para que la señal esté activa **Ton ticks** será: 255-Ton.
  - El valor del timer para que la señal esté inactiva **Toff ticks** será: 255 – T + Ton
  - Asignar a Ton cualquier valor comprendido entre 0 y T para generar los diferentes ciclos de trabajo.

Timer0 y PWM: Ejemplo mejorado (II)

timer0-pwm2.c

```
#include <pic16f876a.h>
#define T 195
void timer0_delay(){...}
void senal(unsigned char Ton)
{
  unsigned int ciclos;
  for (ciclos=0; ciclos<100; ciclos++) {
    PORTB=0xFF;
    timer0_delay(255-Ton);
    PORTB=0x00;
    timer0_delay(255-T+Ton);
  }
}
void main()
{
  TRISB=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=1; PS0=1;
  while(1) {
    senal(39);
    senal(97);
  }
}
```

Periodo del PWM, en ticks

Nueva función para generar PWM. Ahora se le pasa como parámetro Ton (en ticks). Debe estar comprendido entre 0 y 195 (T)

Esperar Ton ticks

Esperar un tiempo de Toff ticks

Prescaler a 256

Señal con ciclo de trabajo del 20%. Este valor se calcula así: 195*0.2 =39

Señal con ciclo de trabajo del 50%. Este valor se calcula así: 195*0.5 =97

Timer0 y PWM: frecuencia VS resolución (I)

- El **timer0** es de 8 bits, por tanto, la máxima resolución se consigue cuando se cuantifica el ancho del pulso en 256 niveles.
- Es decir, que Ton **pueda tomar valores entre 0 y 255**.
- Con ello se consigue la misma resolución que se tendría con un conversor Analógico-digital (DAC) de 8 bits

- **INCONVENIENTE**:

La resolución máxima sólo se consigue para algunas frecuencias

Divisor	T	Frecuencia	T(ticks)
2	102.4 µs	9.8 Khz	256
4	204.8 µs	4.9 Khz	256
8	409.6 µs	2.4 Khz	256
16	819.2 µs	1.2 Khz	256
32	≃ 1.64 ms	609.8 Hz	256
64	≃ 3.28 ms	304.9 Hz	256
128	≃ 6.6 ms	151.5 Hz	256
256	≃ 13.1 ms	76.3 Hz	256

Frecuencias donde se consigue la mayor resolución para el PWM

Timer0 y PWM: frecuencia VS resolución (II)

- Hay un compromiso entre **frecuencia** y **resolución**
- **No podremos generar una señal pwm de la frecuencia que queramos y con la máxima resolución.**
- La máxima resolución sólo se consigue para algunas frecuencias
- Sin embargo, en muchas aplicaciones la frecuencia exacta del PWM no es importante. En estos casos seleccionaremos aquellas en las que la resolución sea máxima.

- El valor de T0INI para que la señal esté un tiempo Toff la hemos calculado como: 255 – T + Ton
- Al trabajar en **máxima resolución**, T=255 y por tanto:

$$255 - T + Ton = Ton$$

Timer0 y PWM: Ejemplo de resolución máxima

timer0-pwm3-fade.c

```
#include <pic16f876a.h>
void timer0_delay() {...}
void senal(unsigned char Ton)
{
  unsigned int ciclos;
  for (ciclos=0; ciclos<10; ciclos++) {
    PORTB=0xFF;
    timer0_delay(255-Ton);
    PORTB=0xF0;
    timer0_delay(Ton);
  }
}
void main(void)
{
  unsigned char i;
  TRISB=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=0; PS0=0;
  for (i=255; i>0; i--) {
    senal(i);
  }
  senal(0);
  while(1);
}
```

Función para generar PWM. Ton debe estar comprendido entre 0 y 255

Esperar a que transcurra un tiempo igual a Ton (ticks)

Sólo se genera el PWM en los 4 bits menos significativos del puerto B. El resto se mantiene encendidos, para compara luminosidades

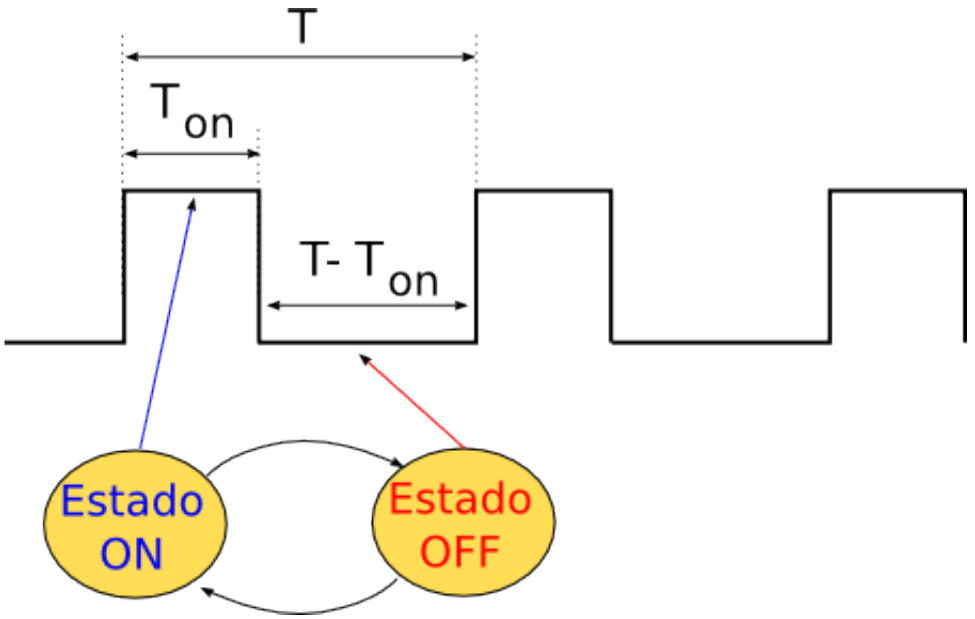
Esperar a que transcurra un tiempo igual a Toff (ticks)

Establecer prescaler. Determina el tiempo que tarda en que la luz de los leds se desvanezca

Decrementar el ciclo de trabajo desde el máximo hasta el mínimo. La luz de los leds se irá desvaneciendo.

PWM Mediante interrupciones (I)

• La generación del **PWM** conviene hacerla mediante **interrupciones** para que el PIC pueda realizar otras **tareas en paralelo**.



• Usamos un **autómata de 2 estados**.

• La rutina de atención a la interrupción configura el timer 0 con el valor correcto según el estado

• Los estados se van alternando

• La comunicación entre el bucle principal y la interrupción es mediante la variable global Ton

PWM Mediante interrupciones (II)

timer0-pwm-int.c (parte I)

```
#include <pic16f876a.h>
#define ESTADO_ON  0
#define ESTADO_OFF 1

unsigned char estado=0;
unsigned char T0ini;
unsigned char Ton=0;
volatile unsigned char cont=0;
void isr() interrupt 0
{
  if (estado==ESTADO_ON) {
    PORTB=0xFF;
    T0ini=255-Ton;
    estado=ESTADO_OFF;
  }
  else {
    PORTB=0x00;
    T0ini=Ton;
    estado=ESTADO_ON;
    cont++;
  }
  TMR0=T0ini;
  TMR0IF=0;
}
```

Definiciones para los estados del autómata

Estado del autómata

Valor inicial del Timer para generar las temporizaciones

Tiempo de activación del pulso. Interfaz entre el bucle principal y la rutina de atención a las interrupciones

Contador de ciclos

Rutina de atención a la interrupción. Se implementa el autómata

Se activan todos los leds durante Ton y se pasa al siguiente estado

En este estado se desactivan los leds durante Toff, se pasa al siguiente estado y se incrementa el contador de ciclos.

Establecer el valor inicial del temporizador para realizar las temporizaciones

Quitar flag de interrupción

PWM Mediante interrupciones (III)

timer0-pwm-int.c (parte II)

```
void main()
{
  unsigned char i;
  TRISB=0;
  T0CS=0; PSA=0;
  PS2=1; PS1=0; PS0=0;
  T0IE=1;
  GIE=1;

  for (i=255; i>0; i--) {
    Ton=i;
    cont=0;
    while(cont<10);
  }
  T0IE=0;
  PORTB=0;
  while(1);
}
```

Este ejemplo hace que el brillo de los leds se vaya desvaneciendo hasta que se apaguen. La señal PWM se genera mediante interrupciones

Activar las interrupciones

Bucle para decrementar el ciclo de trabajo desde el máximo hasta el mínimo. El brillo de los leds se irán desvaneciendo

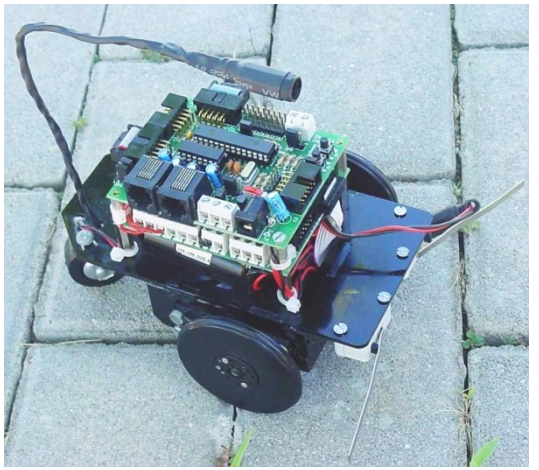
Establecer el valor de Ton para la señal PWM

Poner a cero el contador de ciclos. La interrupción lo incrementa

Esperar a que transcurran 10 ciclos. La señal PWM se genera mediante interrupciones. Aquí el pic podría estar haciendo otra tarea.

Deshabilitar la interrupción

Moviendo el robot Skybot (I)



• El **robot skybot** tiene dos motores de corriente continua (servos trucados)

• Cada motor se controla con 2 bits

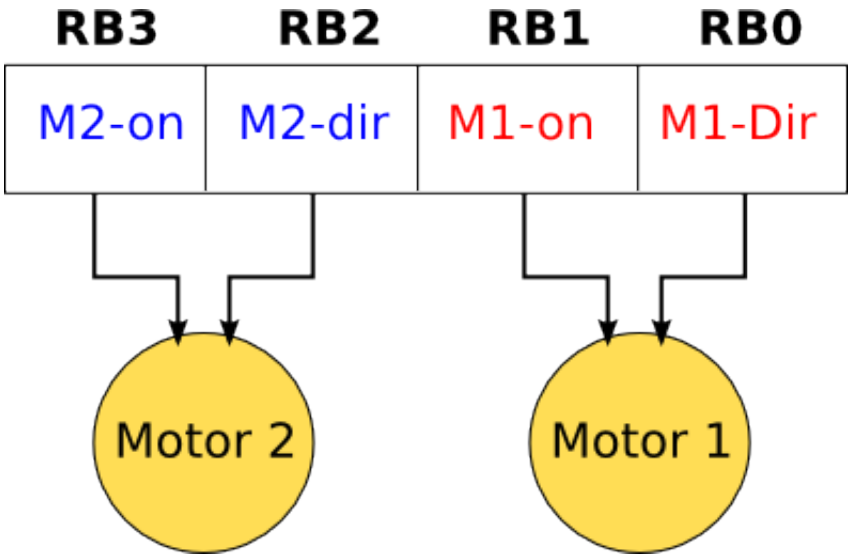
  • Uno indica la dirección de giro

  • Otro si está encendido o apagado

• Vamos a **controlar la velocidad** de estos motores mediante **Pwm**

• Desde un **terminal serie** estableceremos el ciclo del trabajo, desde el 10% hata el 90%, usando las teclas del 1 al 9

• Con O,P,Q,A movemos el robots hacia adelante, atras, izquierda y derecha sin señal PWM (a la máxima velocidad)



Moviendo el robot Skybot (II)

timer0-pwm-sci.c (parte I)

```
#include <pic16f876a.h>
#include "sci.h"

#define M2_ON  RB4
#define M2_DIR RB3
#define M1_ON  RB2
#define M1_DIR RB1

#define ESTADO_ON  0
#define ESTADO_OFF 1

unsigned char ct[]={26,51,
77,103,128,154,179,205,230};

unsigned char estado=0;
unsigned char T0ini;
unsigned char Ton=0;
```

Usar la librería del puerto serie

Definir en qué pines se encuentran los diferentes bits que controlan los motores del skybot

Definición de los estados para el autómata de generación del PWM mediante interrupciones

Valores de Ton para conseguir ciclos de trabajo del 10%, 20% ... hasta el 90%

Estado del autómata

Valor inicial del temporizador

Duración del pulso en ticks

Moviendo el robot Skybot (III)

timer0-pwm-sci.c (parte II)

```
void isr() interrupt 0
{
  if (estado==ESTADO_ON) {
    M1_ON=1;
    M2_ON=1;
    T0ini=255-Ton;
    estado=ESTADO_OFF;
  }
  else {
    M1_ON=0;
    M2_ON=0;
    T0ini=Ton;
    estado=ESTADO_ON;
  }
  TMR0=T0ini;
  TMR0IF=0;
}
```

Rutina de atención a la interrupción

Estado en el que se activa el pulso

Activar las señales de los motores (bits de ON)

Valor inicial del timer para que transcurra un tiempo igual a Ton

Pasar al siguiente estado

Estado en el que se desactiva el pulso

Desactivar las señales de los motores

Valor inicial del timer para que transcurra un tiempo igual a Toff

Pasar al siguiente estado

Asignar valor inicial al timer para que empiece a contar el tiempo

Quitar flag de interrupción

timer0-pwm-sci.c (parte III)

Moviendo el robot Skybot (IV)

```
void main(void)
{
  unsigned char c;
  unsigned char i;

  TRISB=0xE1;
  TOCS=0; PSA=0;
  PS2=1; PS1=0; PS0=0;
  sci_conf();
  GIE=1;

  while(1) {
    c=sci_read();
    if (c>='1' && c<'9') {
      i=c-'1';
      Ton=ct[i];

      M1_DIR=0; M2_DIR=1;
      TMR0IE=1;
    }

(...)
```

Programa principal. Se encarga de recibir los comandos que el usuario envía a través del puerto serie

Configurar puerto B para trabajar con el skybot: 1110 001 --> bits RB4, RB3, RB2 y RB1 son de salida, resto de entradas

Configurar timer0 como temporizador

Divisor a 32

Configurar puerto serie

Activar interrupciones globales (pero no la del timer0)

Bucle principal

Esperar a que se reciba un carácter por el SCI. Las interrupciones mientras tanto estarán generando las señales PWM

Si se recibe cualquiera de las teclas del 1 al 9...

Obtener su índice a la tabla ct. La Tecla '1' será la 0

Establecer el valor de Ton correspondiente al ciclo de trabajo seleccionado

Establecer sentido giro motores para ir hacia adelante

Activar las interrupciones. La señal se empieza a generar

Moviendo el robot Skybot (V)

timer0-pwm-sci.c (parte IV)

```
switch(c) {
     case 'q':
       TMR0IE=0;
       M1_ON=1;  M2_ON=1;
       M1_DIR=0; M2_DIR=1;
       break;
     case 'a':
       TMR0IE=0;
       M1_ON=1;  M2_ON=1;
       M1_DIR=1; M2_DIR=0;
       break;
     case 'o':
       TMR0IE=0;
       M1_ON=1;  M2_ON=1;
       M1_DIR=1; M2_DIR=1;
       break;
```

```
       case 'p':
         TMR0IE=0;
         M1_ON=1;  M2_ON=1;
         M1_DIR=0; M2_DIR=0;
         break;
       case ' ':
         TMR0IE=0; PORTB=0;
         break;
     }
   }
}
```

Con “espacio” se para el robot

En esta parte se procesan las teclas O,P,Q,A para que el robot se mueva a su máxima velocidad, sin usar PWM.

Sólo hay que asignar los valores correctos a los bits de control de los motores.

Ejercicio:

• Modificar el programa **timer0-pwm-sci.c** para que **SÓLO** uno de los motores del **Skybot** se mueva aplicando una señal PWM de 2Khz exactos. Las teclas del 1 al 9 deberán producir ciclos de trabajo del 10% hasta el 90% respectivamente.

Ejercicio final del timer 0

• Hacer un **órgano digital** por el puerto serie (similar al del ejemplo timer0-organo.c)

• Los **tonos** deberán sonar **durante un tiempo** y luego silenciarse.

• Para lograrlo, seguir un efoque similar al del programa timer0-pwm-int.c, donde se crea una variable contador. Expresar la duración en ciclos.

Mejoras:

• Hacer que el tiempo que dura cada nota sea exactamente **medio segundo**

• Añadir nuevas opciones para que el usuario pueda **incrementar** y **decrementar** la duración de las notas